

Lecture 4 - Probability Foundation and Linear Algebra for Machine Learning

Prof. Xiaowei Huang

<https://cgi.csc.liv.ac.uk/~xiaowei/>

(Attendance Code: **755033**)

- For python and other practical issues, please go to the lab sessions and ask the demonstrators directly.
- Anything related to the class/lab schedules, please get in touch with the student office

In the last week's lectures,



Module contents



A few applications of machine learning

Representation of instances as vectors

Data preprocessing

Data



Learning basics

Supervised, unsupervised, semi-supervised learning

ML tasks

Today's Topics

A-level math

Try to recap a bit

Probability distribution

- Joint probability, conditional probability, etc

Two types of probabilistic queries

- Probability query
- MAP

Linear algebra

- Scalars, vectors, matrices, tensors
- Multiplying matrices/vectors
- L_p Norms

Introduction to Scientific Python

- Numpy
- Scipy
- Matplotlib

Lab sessions

Probability foundations



Random Variables



Joint and Conditional Distributions



Independence and Conditional Independence



Querying Joint Probability Distributions

Probability query
MAP query

Machine Learning is about the measurement of, and the reasoning about, probabilistic distributions.



Random Variables

Random Variable

We have a population of students

Use RVs to
express
uncertainty etc

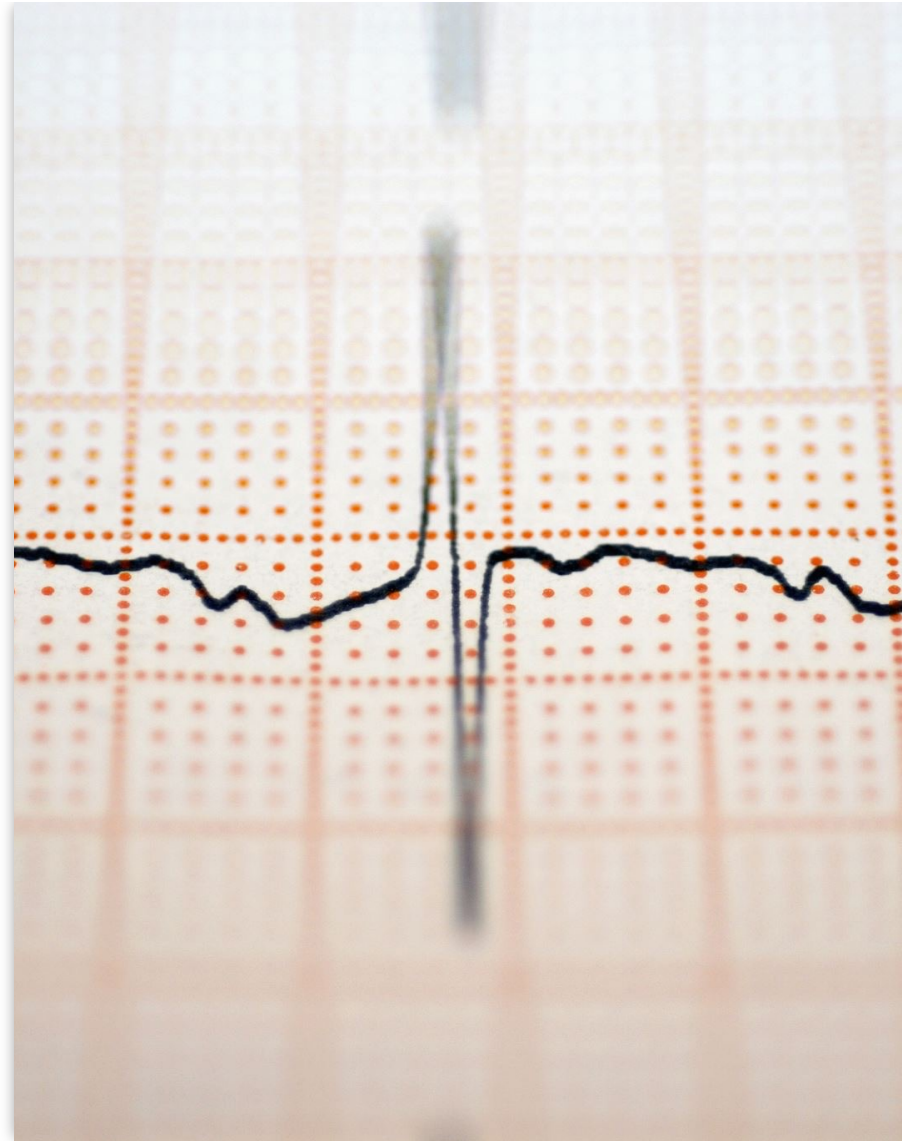
- We want to reason about their grades
- Random variable: *Grade*
- $P(\text{Grade})$ associates a probability with each outcome $\text{Val}(\text{Grade}) = \{A, B, C\}$

If $k = |\text{Val}\{X\}|$ then $\sum_{i=1}^k P(X = x^i) = 1$

- Distribution is referred to as a *multinomial*
- If $\text{Val}\{X\} = \{\text{false}, \text{true}\}$ then it is a *Bernoulli* distribution

$P(X)$ is known as the marginal distribution of X

Joint and Conditional Distributions



Recap: Marginal, joint, conditional probability

- **Marginal probability:** the probability of an event occurring $P(A)$, it may be thought of as an unconditional probability. It is not conditioned on another event.
 - Example: the probability that a card drawn is red, i.e., $P(\text{red}) = 0.5$.
 - Another example: the probability that a card drawn is a 4, i.e., $P(\text{four}) = 1/13$.
- **Joint probability:** $P(A \text{ and } B)$. The probability of event A **and** event B occurring. It is the probability of the intersection of two or more events. The probability of the intersection of A and B may be written $P(A \cap B)$.
 - Example: the probability that a card is a four and red, i.e., $P(\text{four and red}) = 2/52 = 1/26$. (There are two red fours in a deck of 52, the 4 of hearts and the 4 of diamonds).
- **Conditional probability:** $P(A|B)$ is the probability of event A occurring, given that event B occurs.
 - Example: given that you drew a red card, what's the probability that it's a four, i.e., $P(\text{four}|\text{red}) = 2/26 = 1/13$. So out of the 26 red cards (given a red card), there are two fours so $2/26 = 1/13$.

Joint Distribution

- We are interested in questions involving several random variables
 - Example event: *Intelligence=high* and *Grade=A*
 - Need to consider joint distributions
 - Over a set $\chi = \{X_1, \dots, X_n\}$ denoted by $P(X_1, \dots, X_n)$
 - We use ξ to refer to a full assignment to variables χ , i.e. $\xi \in \text{Val}(\chi)$
- Example of joint distribution
 - and marginal distributions

		Intelligence	
		Low	High
Grade	A	0.07	0.18
	B	0.28	0.09
	C	0.35	0.03

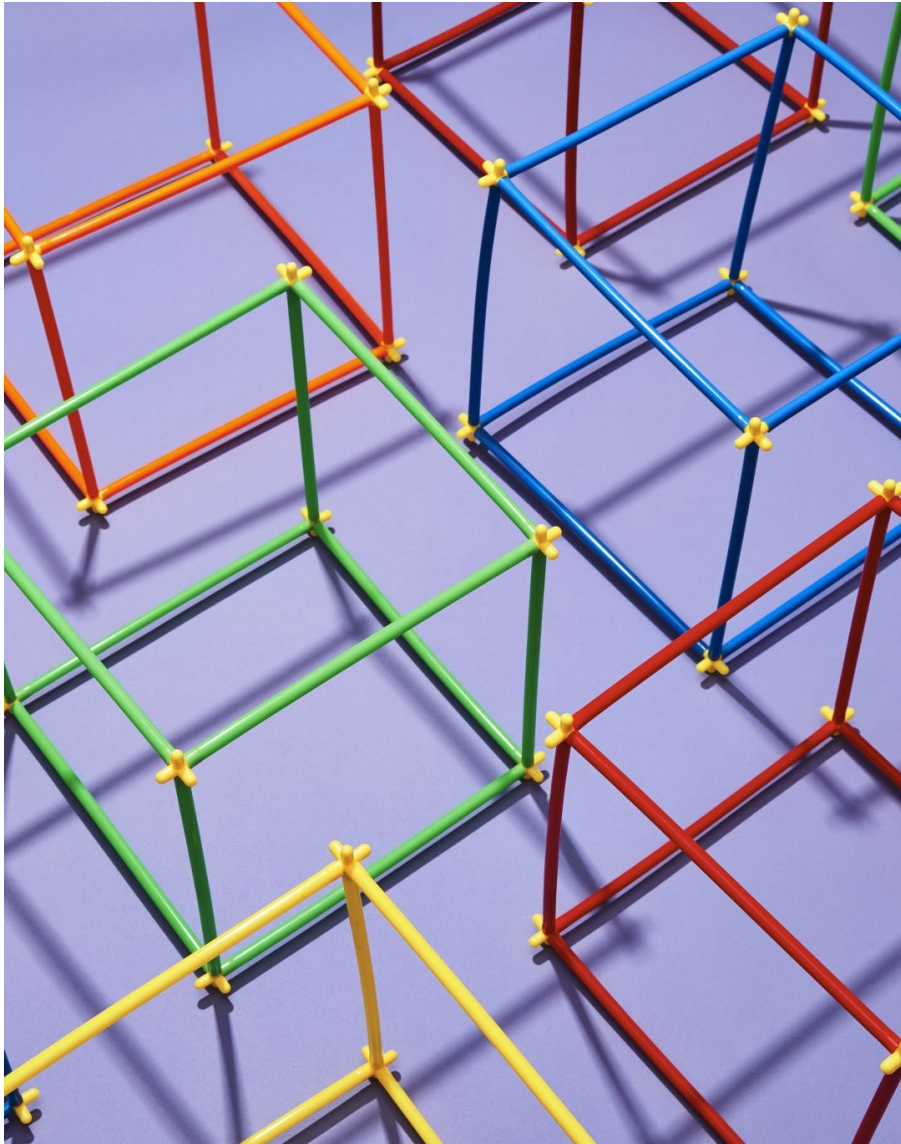
Conditional Probability

- $P(\text{Intelligence}|\text{Grade}=A)$ describes the distribution over events describable by Intelligence given the knowledge that student's grade is A
 - It is not the same as the marginal distribution

$$P(\text{Intelligence}=\text{high})=0.3$$

$$\begin{aligned}P(\text{Intelligence}=\text{high}|\text{Grade}=A) \\ &= 0.18/0.25 \\ &= 0.72\end{aligned}$$

		Intelligence		
		Low	High	
Grade	A	0.07	0.18	0.25
	B	0.28	0.09	0.37
	C	0.35	0.03	0.38
		0.7	0.3	



Independence and Conditional Independence

Recap: Chain Rules

chain rule (also called the **general product rule**) permits the calculation of any member of the joint distribution of a set of random variables using only conditional probabilities.

$$P(A_n, \dots, A_1) = P(A_n | A_{n-1}, \dots, A_1) \cdot P(A_{n-1}, \dots, A_1)$$

$$P(A_4, A_3, A_2, A_1) = P(A_4 | A_3, A_2, A_1) \cdot P(A_3 | A_2, A_1) \cdot P(A_2 | A_1) \cdot P(A_1)$$

Independent Random Variables

We expect $P(\alpha | \beta)$ to be different from $P(\alpha)$

- i.e., β is true changes our probability over α

Sometimes equality can occur, i.e., $P(\alpha | \beta) = P(\alpha)$

- i.e., learning that β occurs did not change our probability of α

We say event α is **independent** of event β , denoted

- $\alpha \perp \beta$, if $P(\alpha | \beta) = P(\alpha)$ or if $P(\beta) = 0$

A distribution P satisfies $(\alpha \perp \beta)$ if and only if $P(\alpha \wedge \beta) = P(\alpha)P(\beta)$



Conditional Independence

- While independence is a useful property, we don't often encounter two independent events
- A more common situation is when two events are independent given an additional event
 - Reason about student accepted at Stanford or MIT
 - These two are not independent
 - If student admitted to Stanford then probability of MIT is higher
 - If both based on GPA and we know the GPA to be A
 - Then the student being admitted to Stanford does not change probability of being admitted to MIT
 - $P(\text{MIT} | \text{Stanford}, \text{Grade A}) = P(\text{MIT} | \text{Grade A})$
 - i.e., MIT is conditionally independent of Stanford given Grade A

Querying Joint Probability Distributions



Query Types

Probability Queries

- Given evidence (the values of a subset of random variables),
- compute distribution of another subset of random variables

As a quick/intuitive understanding:

The best cell

Grade

Intelligence

The best combination

	Low	High
A	0.07	0.18
B	0.28	0.09
C	0.35	0.03

MAP Queries

- Maximum a posteriori probability
- Also called MPE (*Most Probable Explanation*)
 - What is the most likely setting of a subset of random variables
- Marginal MAP Queries
 - When some variables are known

Probability Queries -- Most common type of queries



Query has two parts

Evidence: a subset E of variables and their instantiation e

Query Variables: a subset Y of random variables



Inference Task: $P(Y/E=e)$

Posterior probability distribution over values y of Y

Conditioned on the fact $E=e$

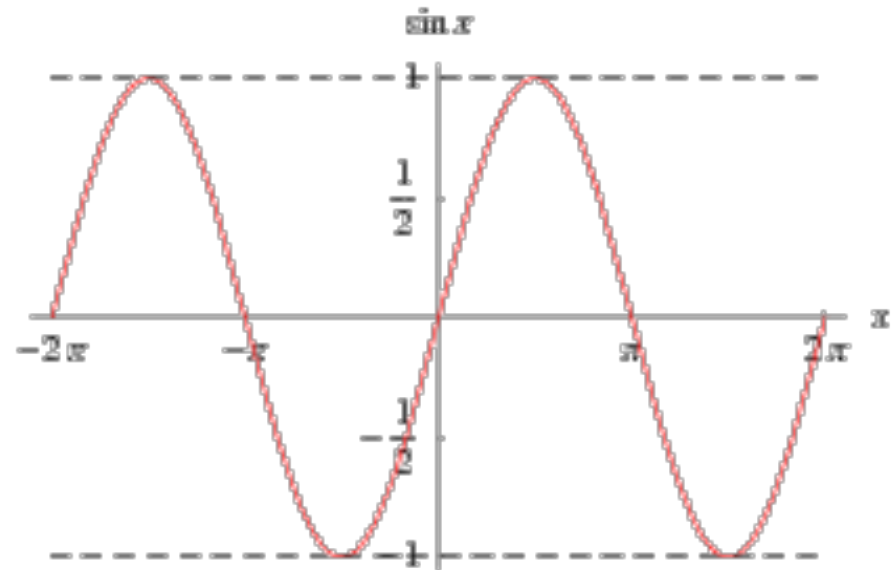
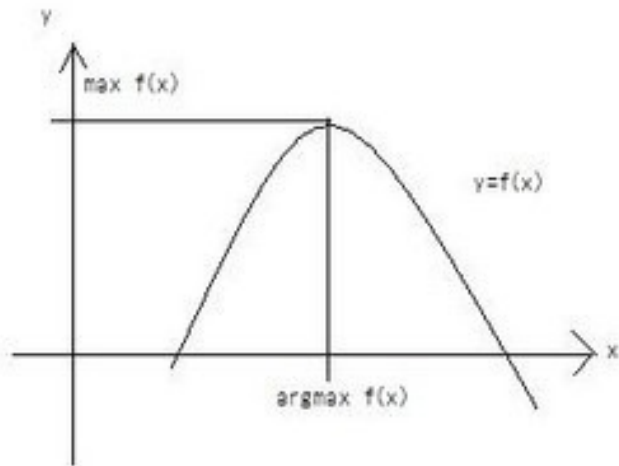
Can be viewed as Marginal over Y in distribution we obtain by conditioning on e

Probability Queries

- Marginal Probability Estimation

$$P(Y = y_i | E = e) = \frac{P(Y = y_i, E = e)}{P(E = e)}$$

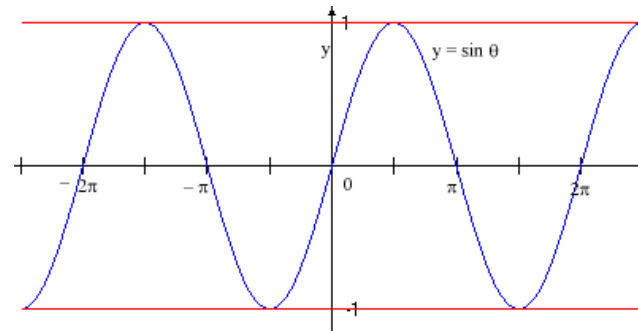
Recap: max and argmax



Recap: Max vs. argmax

- Let x be in a range $[a,b]$ and f be a function over $[a,b]$, we have
 - $\max f(x)$ to represent the maximum value of $f(x)$ as x varies through $[a,b]$
 - $\operatorname{argmax} f(x)$ to represent the value of x at which the maximum is attained

- $\max_x \sin(x)$
 $= 1$
- $\operatorname{argmax}_x \sin(x)$
 $= \{(0.5+2n)*\pi \mid n \text{ is integer} \}$
 $= \{..., -1.5\pi, 0.5\pi, 2.5\pi, ...\}$



MAP Queries (Most Probable Explanation)

- Finding a high probability assignment to some subset of variables
- Most likely assignment to all non-evidence variables $W = V - E$

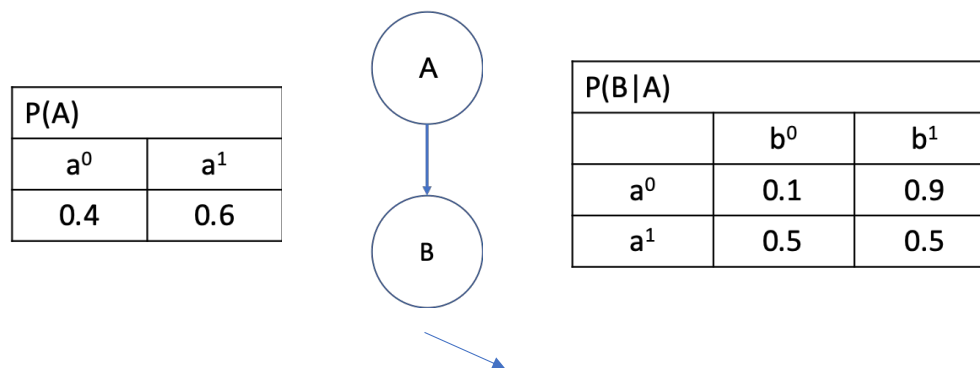
$$MAP(W | e) = \arg \max_w P(w, e)$$

i.e., value of w for which $P(w, e)$ is maximum

- Difference from probability query
 - Instead of a probability we get the most likely configuration (combination of variable values) for all remaining variables

Example of MAP Queries

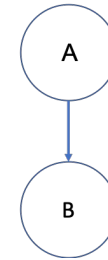
- Medical Diagnosis Problem
 - Diseases (A) cause Symptoms (B)
 - Two possible diseases: Mono and Flu
 - Two possible symptoms: Headache and Fever



Notation for probabilistic graphical models, to be introduced in later part of this module

Example of MAP Queries

- Medical Diagnosis Problem
 - Diseases (A) cause Symptoms (B)
 - Two possible diseases: Mono and Flu
 - Two possible symptoms: Headache and Fever
- Q1: Most likely disease $MAP(A)$?
- Q2: Most likely disease and symptom $MAP(A,B)$?
- Q3: Most likely symptom $MAP(B)$?



P(A)	
a^0	a^1
0.4	0.6

P(B A)		
	b^0	b^1
a^0	0.1	0.9
a^1	0.5	0.5

Example of MAP Queries

- Medical Diagnosis Problem
 - Diseases (A) cause Symptoms (B)
 - Two possible diseases: Mono and Flu
 - Two possible symptoms: Headache and Fever
- Q1: Most likely disease $MAP(A)$?

$$MAP(A) = \arg \max_a A = a^1$$

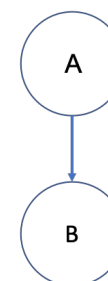
P(A)	
a^0	a^1
0.4	0.6

Example of MAP Queries

- Medical Diagnosis Problem
 - Diseases (A) cause Symptoms (B)
 - Two possible diseases: Mono and Flu
 - Two possible symptoms: Headache and Fever
- Q2: Most likely disease and symptom $P(A,B)$?

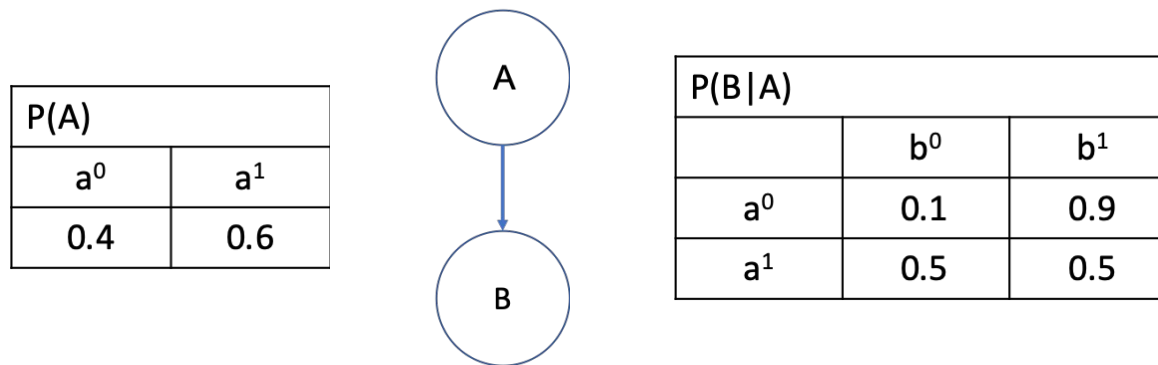
$$MAP(A, B) = \arg \max_{a,b} P(A, B)$$

P(A)	
a^0	a^1
0.4	0.6



P(B A)		
	b^0	b^1
a^0	0.1	0.9
a^1	0.5	0.5

Example of MAP Queries



$$P(A,B) = P(B|A) P(A)$$

P(A,B)		
	b^0	b^1
a^0	0.04	0.36
a^1	0.3	0.3

Example of MAP Queries

- Medical Diagnosis Problem
 - Diseases (A) cause Symptoms (B)
 - Two possible diseases: Mono and Flu
 - Two possible symptoms: Headache and Fever
- Q2: Most likely disease and symptom $P(A,B)$?

$$\begin{aligned} MAP(A, B) &= \arg \max_{a,b} P(A, B) \\ &= \arg \max_{a,b} P(B \mid A)P(A) \\ &= \arg \max_{a,b} \{0.04, 0.36, 0.3, 0.3\} \\ &= a^0, b^1 \end{aligned}$$

P(A)	
a^0	a^1
0.4	0.6

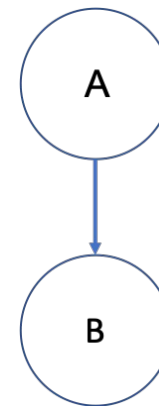


P(B A)		
	b^0	b^1
a^0	0.1	0.9
a^1	0.5	0.5

In this case, $Y = \{A, B\}$, $E = \{\}$, $Z = \{\}$, where Y, Z, E are defined in the next slide.

Example of MAP Queries

- Q3: Most likely symptom $MAP(B)$?



P(A)	
a^0	a^1
0.4	0.6

P(B A)		
	b^0	b^1
a^0	0.1	0.9
a^1	0.5	0.5

MAP Query

- Y is query, evidence is $E=e$. Task is to find most likely assignment to Y :

- If $Z=X-Y-E$ is empty

$$MAP(Y | e) = \arg \max_y P(y | e)$$

- If $Z=X-Y-E$ is not empty

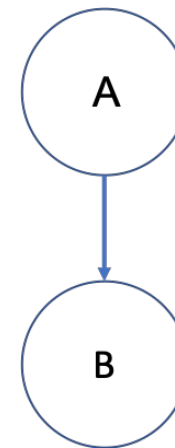
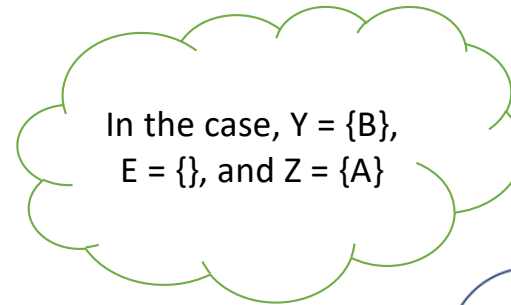
$$MAP(Y | e) = \arg \max_y \sum_z P(y, z | e)$$

The set X of features can be split into three subsets Y , E , and Z , where

- Y is the set of variables to reason about,
- E is the set of variables that we observe, and
- Z is the set of variables we don't care.

Example of MAP Queries

- Medical Diagnosis Problem
 - Diseases (A) cause Symptoms (B)
 - Two possible diseases: Mono and Flu
 - Two possible symptoms: Headache and Fever
- Q3: Most likely symptom $P(B)$?



P(A)	
a^0	a^1
0.4	0.6

P(B A)		
	b^0	b^1
a^0	0.1	0.9
a^1	0.5	0.5

Example of MAP Queries

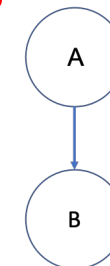
definition

- Q3: Most likely symptom $P(B)$?

$$\begin{aligned} \text{MAP}(B) &= \arg \max_b P(b) = \arg \max_b \sum_a P(a, b) \\ &= \arg \max_b \{0.34, 0.66\} = b^1 \end{aligned}$$

marginalisation

P(A)	
a^0	a^1
0.4	0.6



$$P(A, B) = P(A)P(B|A)$$

P(A, B)		
	b^0	b^1
a^0	0.04	0.36
a^1	0.3	0.3

P(B A)		
	b^0	b^1
a^0	0.1	0.9
a^1	0.5	0.5

Exercise 1

Joint distribution table as shown right

[Check the exercises of the lecture notes
for answer](#)

Can you calculate the following:

$$P(A=1) = 0.6$$

$$P(A=2) = 0.3$$

$$P(B=3) = 0.4$$

$$P(B=4) = 0.1$$

$$P(A=1 | B=2) = 0.6$$

$$P(B=3 | A=3) = 0.4$$

$$\text{MAP}(A | B=2) = 1$$

$$\text{MAP}(B | A=2) = 3$$

$$\text{MAP}(A) = 1$$

$$\text{MAP}(B) = 3$$

	B=1	B=2	B=3	B=4
A=1	0.12	0.18	0.24	0.06
A=2	0.06	0.09	0.12	0.03
A=3	0.02	0.03	0.04	0.01

Exercise 2

[Check the exercises of the lecture notes
for answer](#)

Joint distribution table as shown right

Can you calculate the following:

$$P(A=1) = 0.56$$

$$P(A=2) = 0.3$$

$$P(B=3) = 0.4$$

$$P(B=4) = 0.06$$

$$P(A=1 | B=2) = 0.6$$

$$P(B=3 | A=3) = 2/7$$

$$\text{MAP}(A | B=2) = 1$$

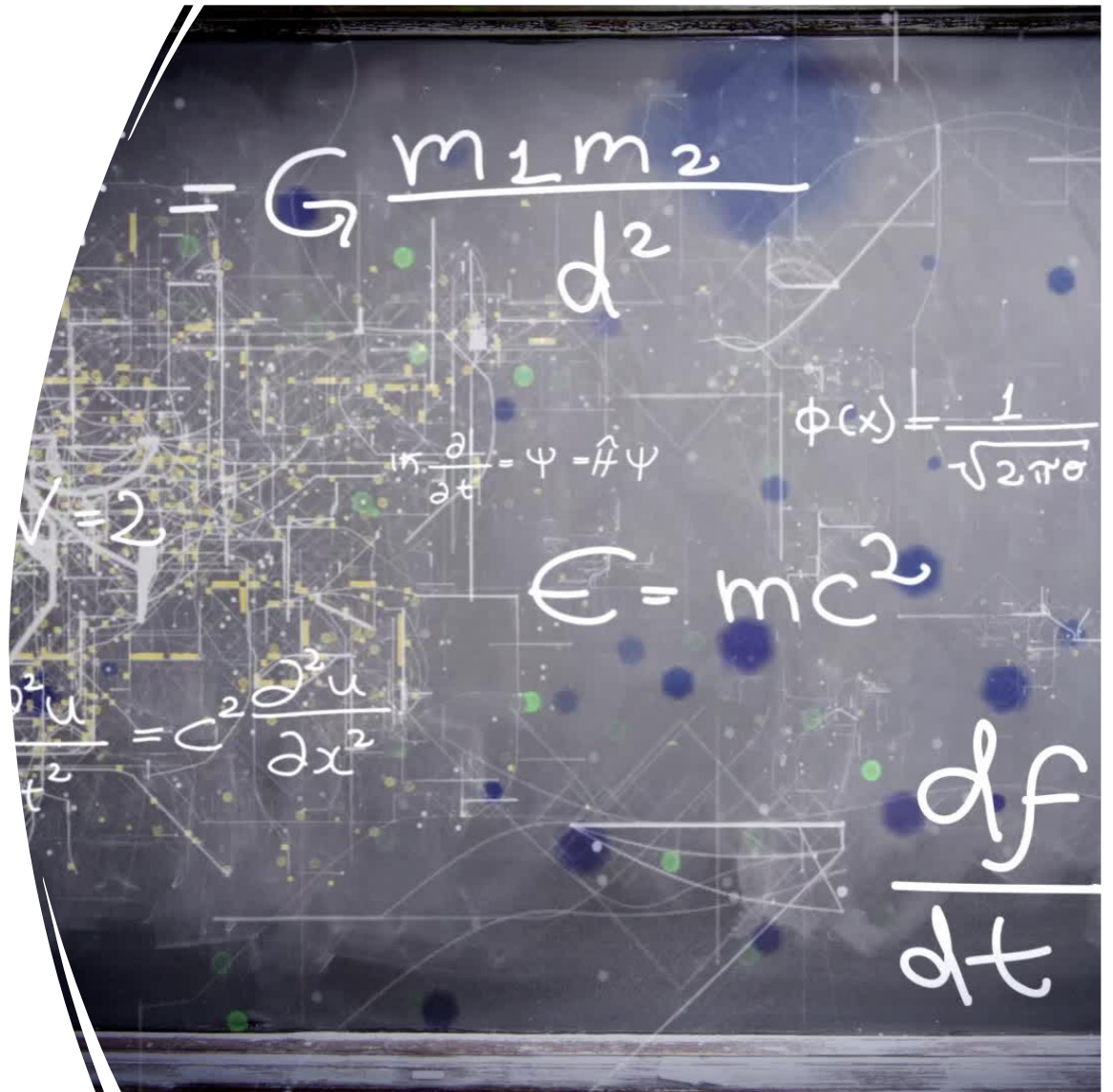
$$\text{MAP}(B | A=2) = 3$$

$$\text{MAP}(A) = 1$$

$$\text{MAP}(B) = 3$$

	B=1	B=2	B=3	B=4
A=1	0.12	0.18	0.24	0.02
A=2	0.06	0.09	0.12	0.03
A=3	0.06	0.03	0.04	0.01

Linear Algebra For Machine Learning



Scalar

- Single number
- Represented in lower-case italic x
 - E.g., let $x \in \mathbb{R}$ be the slope of the line
 - Defining a real-valued scalar
 - E.g., let $n \in \mathbb{N}$ be the number of units
 - Defining a natural number scalar

Vector

- An array of numbers
- Arranged in order
- Each no. identified by an index
- Vectors are shown in lower-case bold
- If each element is in R then $\mathbf{x}$ is in R^n
- We think of vectors as points in space
 - Each element gives coordinate along an axis

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_n \end{bmatrix} \Rightarrow \mathbf{x}^T = [x_1, x_2, \dots, x_n]$$

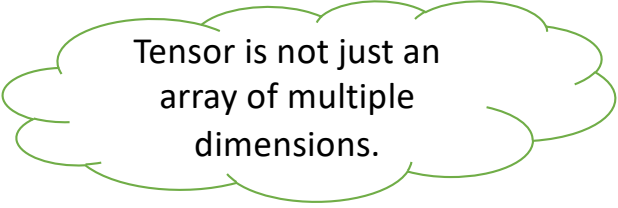
Matrix

- 2-D array of numbers
- Each element identified by two indices
- Denoted by bold typeface **A**
- Elements indicated as $A_{m,n}$
 - E.g.,
- $A[i:]$ is i th row of A , $A[:,j]$ is j th column of **A**
- If A has **shape** of height m and width n with real-values then

$$\mathbf{A} = \mathbb{R}^{m \times n}$$

$$\mathbf{A} = \begin{bmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{bmatrix}$$

Tensor



Tensor is not just an array of multiple dimensions.

- Sometimes need an array with more than two axes (or dimensions)
- An array arranged on a regular grid with variable number of axes is referred to as a tensor
- Denote a tensor with bold typeface: $\mathbf{A}$
- Element (i,j,k) of tensor denoted by $A_{i,j,k}$

Transpose of a Matrix

- Mirror image across principal diagonal

$$A = \begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{bmatrix} \Rightarrow A^T = \begin{bmatrix} x_{11} & x_{21} & x_{31} \\ x_{12} & x_{22} & x_{32} \\ x_{13} & x_{23} & x_{33} \end{bmatrix}$$

- Vectors are matrices with a single column
 - Often written in-line using transpose

$$\mathbf{x} = [x_1, \dots, x_n]^T$$

- Since a scalar is a matrix with one element $a = a^T$

Linear Transformation

$$A\mathbf{x} = \mathbf{b}$$

• where $A \in \mathbb{R}^{n \times n}$ and $\mathbf{b} \in \mathbb{R}^n$

$$A_{11}x_1 + A_{12}x_2 + \dots + A_{1n}x_n = b_1$$

$$A_{21}x_1 + A_{22}x_2 + \dots + A_{2n}x_n = b_2$$

...

$$A_{n1}x_1 + A_{n2}x_2 + \dots + A_{nn}x_n = b_n$$



n equations in
 n unknowns

Linear Transformation

$$A\mathbf{x} = \mathbf{b}$$

- where $A \in \mathbb{R}^{n \times n}$ and $\mathbf{b} \in \mathbb{R}^n$
- More explicitly

$$A = \begin{bmatrix} A_{1,1} & \cdots & A_{1,n} \\ \vdots & \vdots & \vdots \\ A_{n,1} & \cdots & A_{nn} \end{bmatrix} \quad \mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} \quad \mathbf{b} = \begin{bmatrix} b_1 \\ \vdots \\ b_n \end{bmatrix}$$

$n \times n$ $n \times 1$ $n \times 1$

Can view A as a *linear transformation* of vector $\mathbf{x}$ to vector $\mathbf{b}$

- Sometimes we wish to solve for the unknowns $\mathbf{x} = \{x_1, \dots, x_n\}$ when A and $\mathbf{b}$ provide constraints

Identity and Inverse Matrices

- Matrix inversion is a powerful tool to analytically solve $A\mathbf{x}=\mathbf{b}$
- Needs concept of Identity matrix
- Identity matrix does not change value of vector
- when we multiply the vector by identity matrix
 - Denote identity matrix that preserves n-dimensional vectors as I_n
 - Formally $I_n \in \mathbb{R}^{n \times n}$ and $\forall \mathbf{x} \in \mathbb{R}^n, I_n \mathbf{x} = \mathbf{x}$
 - Example of I_3

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Matrix Inverse

- Inverse of square matrix A defined as $A^{-1}A=I_n$
- We can now solve $A\mathbf{x}=\mathbf{b}$ as follows:

$$A\mathbf{x}=\mathbf{b}$$

$$A^{-1}A\mathbf{x} = A^{-1}\mathbf{b}$$

$$I_n \mathbf{x} = A^{-1}\mathbf{b}$$

$$\mathbf{x} = A^{-1}\mathbf{b}$$

- This depends on being able to find A^{-1}
- If A^{-1} exists there are several methods for finding it

Solving Simultaneous equations

- $A\mathbf{x} = \mathbf{b}$
- Two closed-form solutions
 - Matrix inversion $\mathbf{x} = A^{-1}\mathbf{b}$
 - Gaussian elimination

Norms

- Used for measuring the size of a vector
- Norms map vectors to non-negative values
- Norm of vector $\mathbf{x}$ is distance from origin to $\mathbf{x}$
 - It is any function f that satisfies:

$$\begin{aligned} f(\mathbf{x}) = 0 &\Rightarrow \mathbf{x} = \mathbf{0} \\ f(\mathbf{x} + \mathbf{y}) &\leq f(\mathbf{x}) + f(\mathbf{y}) \quad \text{Triangle Inequality} \\ \forall \alpha \in \mathbb{R} \quad f(\alpha \mathbf{x}) &= |\alpha| f(\mathbf{x}) \end{aligned}$$

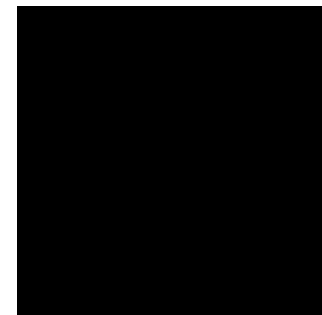
Image distance



-



=



$$\begin{bmatrix} x_{1,1} & x_{1,2} & \dots & x_{1,32} \\ x_{2,1} & x_{2,2} & \dots & x_{2,32} \\ \vdots & \vdots & \ddots & \vdots \\ x_{32,1} & x_{32,2} & \dots & x_{32,32} \end{bmatrix}$$

-

$$\begin{bmatrix} y_{1,1} & y_{1,2} & \dots & y_{1,32} \\ y_{2,1} & y_{2,2} & \dots & y_{2,32} \\ \vdots & \vdots & \ddots & \vdots \\ y_{32,1} & y_{32,2} & \dots & y_{32,32} \end{bmatrix}$$

=

$$\begin{bmatrix} z_{1,1} & z_{1,2} & \dots & z_{1,32} \\ z_{2,1} & z_{2,2} & \dots & z_{2,32} \\ \vdots & \vdots & \ddots & \vdots \\ z_{32,1} & z_{32,2} & \dots & z_{32,32} \end{bmatrix}$$

L^p Norm

- Definition
$$||x||_p = \left(\sum_i |x_i|^p \right)^{\frac{1}{p}}$$

- Definition $||x||_p = \left(\sum_i |x_i|^p \right)^{\frac{1}{p}}$
- L^2 Norm
 - Called Euclidean norm, written simply as $||x||$
 - Squared Euclidean norm is same as $x^T x$

$$\begin{aligned} ||x||_2 &= \sqrt{\sum_i |x_i|^2} \\ &= \sqrt{x^T x} \end{aligned}$$

L^p Norm

- Definition $||x||_p = \left(\sum_i |x_i|^p \right)^{\frac{1}{p}}$
- L^1 Norm
 - also called Manhattan distance

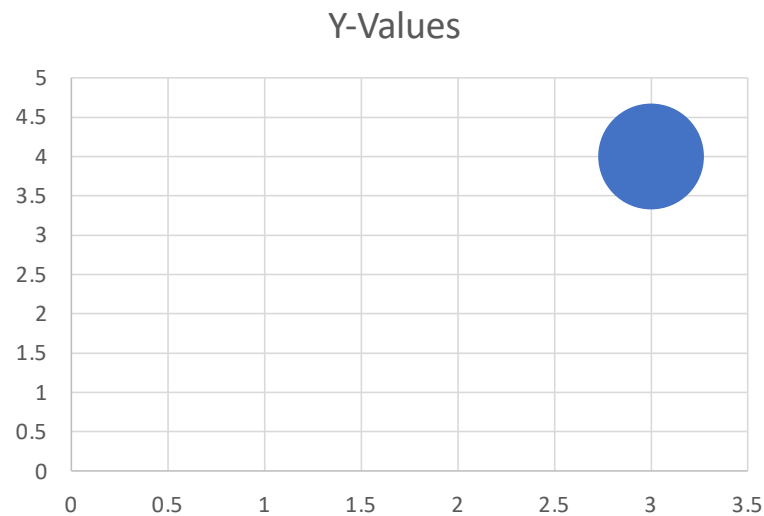
$$||x||_1 = \sum_i |x_i|$$

L^p Norm

- Definition $\|x\|_p = \left(\sum_i |x_i|^p \right)^{\frac{1}{p}}$
- L^∞ Norm
 - also called max norm

$$\|x\|_\infty = \max_i |x_i|$$

Norms of two-dimensional Point



$$X = (3, 4)$$

$$||x||_1 = 3 + 4 = 7$$

$$||x||_2 = \sqrt{3^2 + 4^2} = 5$$

$$||x||_{\infty} = \max\{3, 4\} = 4$$

$$||x||_1 = \sum_i |x_i|$$

$$||x||_2 = \sqrt{\sum_i |x_i|^2}$$

$$||x||_{\infty} = \max_i |x_i|$$

Image distance



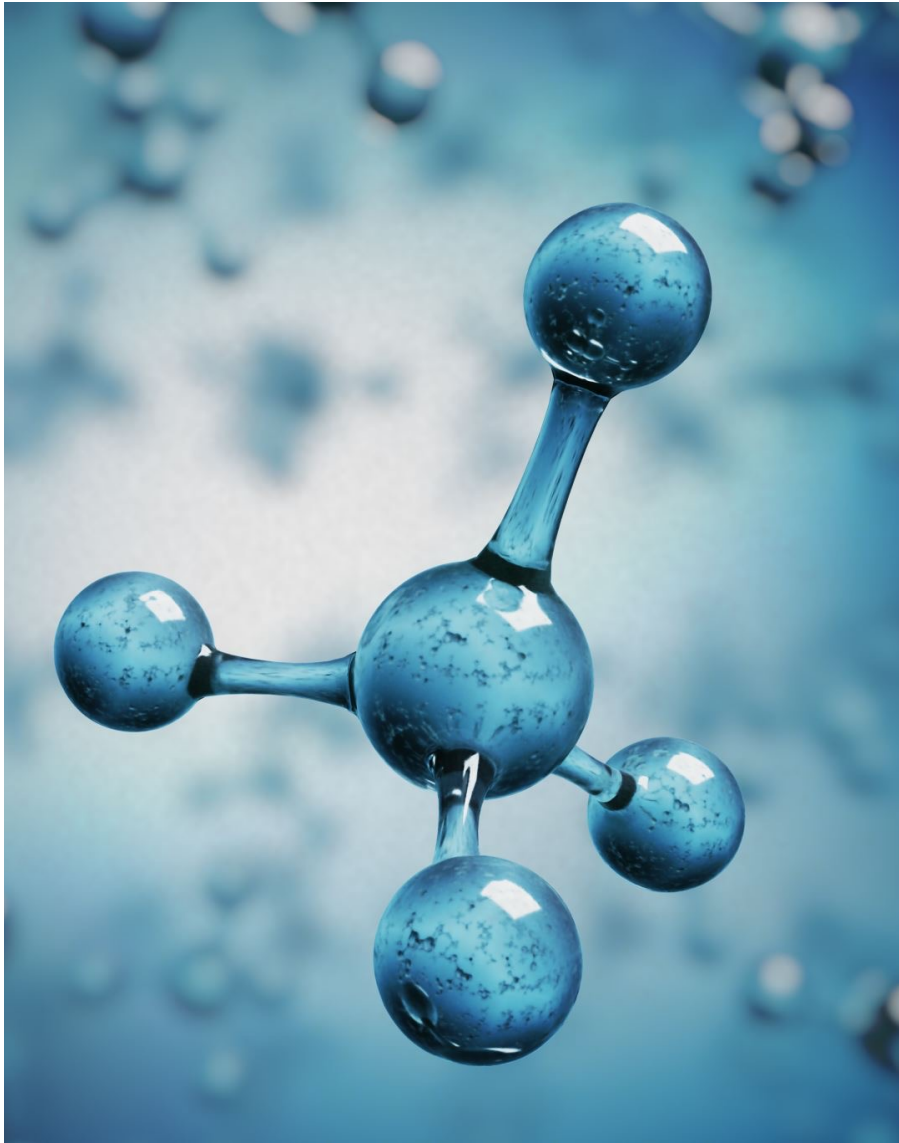
$$\begin{bmatrix} x_{1,1} & x_{1,2} & \dots & x_{1,32} \\ x_{2,1} & x_{2,2} & \dots & x_{2,32} \\ \vdots & \vdots & \ddots & \vdots \\ x_{32,1} & x_{32,2} & \dots & x_{32,32} \end{bmatrix} - \begin{bmatrix} y_{1,1} & y_{1,2} & \dots & y_{1,32} \\ y_{2,1} & y_{2,2} & \dots & y_{2,32} \\ \vdots & \vdots & \ddots & \vdots \\ y_{32,1} & y_{32,2} & \dots & y_{32,32} \end{bmatrix} = \begin{bmatrix} z_{1,1} & z_{1,2} & \dots & z_{1,32} \\ z_{2,1} & z_{2,2} & \dots & z_{2,32} \\ \vdots & \vdots & \ddots & \vdots \\ z_{32,1} & z_{32,2} & \dots & z_{32,32} \end{bmatrix}$$

L^1 distance between X and Y: $\sum_{i,j} |z_{i,j}| = \sum_{i,j} |x_{i,j} - y_{i,j}|$

L^2 distance between X and Y: $\sqrt{\sum_{i,j} z_{i,j}^2} = \sqrt{\sum_{i,j} (x_{i,j} - y_{i,j})^2}$

L^∞ distance between X and Y:

$$\max_{i,j} |z_{i,j}| = \max_{i,j} |x_{i,j} - y_{i,j}|$$



Introduction to Scientific Python



Numpy

- Fundamental package for scientific computing with Python
- N-dimensional array object
- Linear algebra, Fourier transform, random number capabilities
- Building block for other packages (e.g. Scipy)
- Open source



import numpy as np

- Basics:

```
import numpy as np

A = np.array([[1, 2, 3], [4, 5, 6]])
print A
# [[1 2 3]
#  [4 5 6]]

Af = np.array([1, 2, 3], float)
```

- Slicing as usual

More basics

evenly spaced values within a given interval.

```
np.arange(0, 1, 0.2)
# array([ 0. ,  0.2,  0.4,  0.6,  0.8])

np.linspace(0, 2*np.pi, 4)
# array([ 0.0,  2.09,  4.18,  6.28])

A = np.zeros((2,3))
# array([[ 0.,  0.,  0.],
#        [ 0.,  0.,  0.]])
# np.ones, np.diag
A.shape
# (2, 3)
```

numpy.linspace:

evenly spaced numbers over a specified interval.

```
np.random.random((2,3))
# array([[ 0.78084261,  0.64328818,  0.55380341],
#        [ 0.24611092,  0.37011213,  0.83313416]])

a = np.random.normal(loc=1.0, scale=2.0, size=(2,2))
# array([[ 2.87799514,  0.6284259 ],
#        [ 3.10683164,  2.05324587]])

np.savetxt("a_out.txt", a)
# save to file
b = np.loadtxt("a_out.txt")
# read from file
```

More basics

- `numpy.random.normal`:
- Draw random samples from a normal (Gaussian) distribution
- `loc`: mean
- `scale`: standard deviation

Arrays are mutable

```
A = np.zeros((2, 2))  
# array([[ 0.,  0.],  
#        [ 0.,  0.]])  
C = A  
C[0, 0] = 1  
  
print A  
# [[ 1.  0.]  
#   [ 0.  0.]
```

Array attributes

```
a = np.arange(10).reshape((2,5))  
  
a.ndim      # 2 dimension  
a.shape     # (2, 5) shape of array  
a.size      # 10 # of elements  
a.T         # transpose  
a.dtype     # data type
```

- `numpy.reshape`:
- Gives a new shape to an array without changing its data.

Basic operations

- Arithmetic operators: elementwise application

```
a = np.arange(4)
# array([0, 1, 2, 3])

b = np.array([2, 3, 2, 4])

a * b # array([ 0,  3,  4, 12])
b - a # array([2, 2, 0, 1])

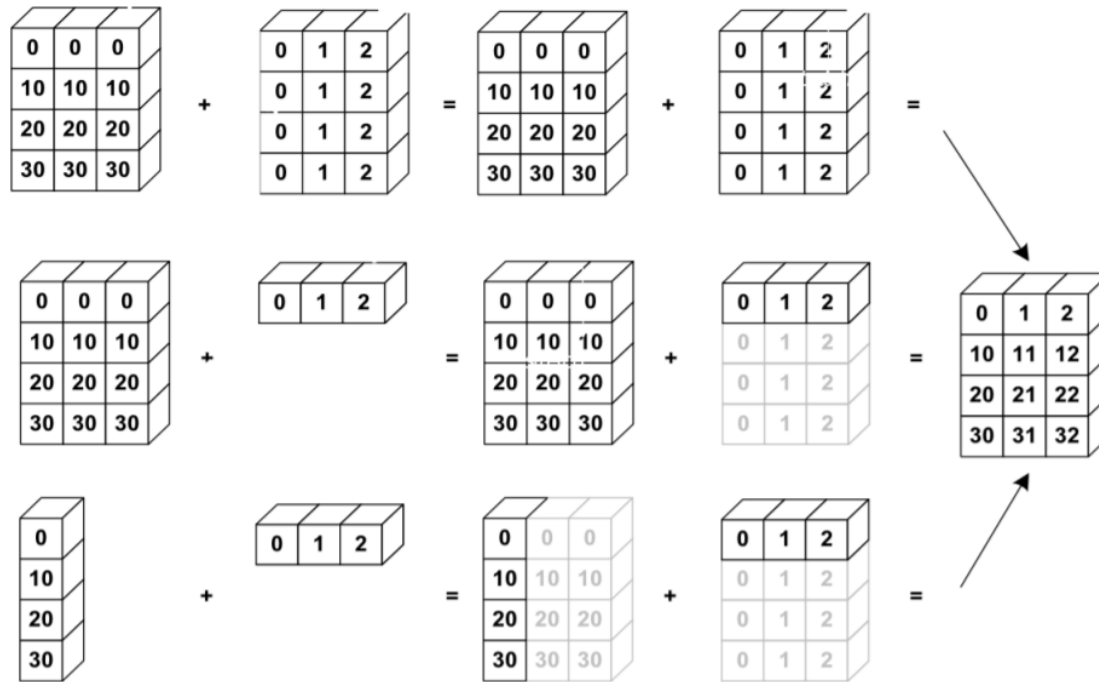
c = [2, 3, 4, 5]
a * c # array([ 0,  3,  8, 15])
```

- Also, we can use += and *=.

Array broadcasting

- When operating on two arrays, numpy compares shapes. Two dimensions are compatible when
 - They are of equal size
 - One of them is 1

Array broadcasting



Array broadcasting with scalars

- This also allows us to add a constant to a matrix or multiply a matrix by a constant

```
A = np.ones((3,3))
```

```
print 3 * A - 1
```

```
# [[ 2.  2.  2.]
```

```
# [ 2.  2.  2.]
```

```
# [ 2.  2.  2.]]
```


Vector operations

- inner product
- outer product
- dot product (matrix multiplication)

```
# note: numpy automatically converts lists
u = [1, 2, 3]
v = [1, 1, 1]

np.inner(u, v)
# 6
np.outer(u, v)
# array([[1, 1, 1],
#        [2, 2, 2],
#        [3, 3, 3]])
np.dot(u, v)
# 6
```

Matrix operations

- First, define some matrices:

```
A = np.ones((3, 2))  
# array([[ 1.,  1.],  
#        [ 1.,  1.],  
#        [ 1.,  1.]])  
A.T  
# array([[ 1.,  1.,  1.],  
#        [ 1.,  1.,  1.]])  
  
B = np.ones((2, 3))  
# array([[ 1.,  1.,  1.],  
#        [ 1.,  1.,  1.]])
```

Matrix operations

- `np.dot(a,b)`
- If both a and b are 1-D arrays, it is inner product of vectors
- If both a and b are 2-D arrays, it is matrix multiplication

```
np.dot(A, B)
# array([[ 2.,  2.,  2.],
#        [ 2.,  2.,  2.],
#        [ 2.,  2.,  2.]])

np.dot(B, A)
# array([[ 3.,  3.],
#        [ 3.,  3.]])

np.dot(B.T, A.T)
# array([[ 2.,  2.,  2.],
#        [ 2.,  2.,  2.],
#        [ 2.,  2.,  2.]])

np.dot(A, B.T)
# Traceback (most recent call last):
#   File "<stdin>", line 1, in <module>
# ValueError: shapes (3,2) and (3,2) not aligned: ...
# ... 2 (dim 1) != 3 (dim 0)
```



Operations along axes

```
a = np.random.random((2,3))
# array([[ 0.9190687 ,  0.36497813,  0.75644216],
#        [ 0.91938241,  0.08599547,  0.49544003]])
a.sum()
# 3.5413068994445549
a.sum(axis=0) # column sum
# array([ 1.83845111,  0.4509736 ,  1.25188219])
a.cumsum()
# array([ 0.9190687 ,  1.28404683,  2.04048899,  2.9598714 ,
#        [ 3.04586687,  3.5413069 ]])
a.cumsum(axis=1) # cumulative row sum
# array([[ 0.9190687 ,  1.28404683,  2.04048899],
#        [ 0.91938241,  1.00537788,  1.50081791]])
a.min()
# 0.0859954690403677
a.max(axis=0)
# array([ 0.91938241,  0.36497813,  0.75644216])
```

Slicing arrays

```
a = np.random.random((4,5))
```

```
a[2, :]
```

```
# third row, all columns
```

```
a[1:3]
```

```
# 2nd, 3rd row, all columns
```

```
a[:, 2:4]
```

```
# all rows, columns 3 and 4
```

- Iterating over multidimensional arrays is done with respect to the first axis: **for row in A**
- Looping over all elements: **for element in A.flat**

- Reshape
 - using reshape. Total size must remain the same.
- Resize
 - using resize, always works: chopping or appending zeros
 - First dimension has 'priority', so beware of unexpected results

Linear algebra

```
import numpy.linalg
```

<code>qr</code>	Computes the QR decomposition
<code>cholesky</code>	Computes the Cholesky decomposition
<code>inv(A)</code>	Inverse
<code>solve(A,b)</code>	Solves $Ax = b$ for A full rank
<code>lstsq(A,b)</code>	Solves $\arg \min_x \ Ax - b\ _2$
<code>eig(A)</code>	Eigenvalue decomposition
<code>eig(A)</code>	Eigenvalue decomposition for symmetric or hermitian
<code>eigvals(A)</code>	Computes eigenvalues.
<code>svd(A, full)</code>	Singular value decomposition
<code>pinv(A)</code>	Computes pseudo-inverse of A

Random sampling

```
import numpy.random
```

```
rand(d0,d1,...,dn)
```

Random values in a given shape

```
randn(d0, d1, ...,dn)
```

Random standard normal

```
randint(lo, hi, size)
```

Random integers [lo, hi)

```
choice(a, size, repl, p)
```

Sample from a

```
shuffle(a)
```

Permutation (in-place)

```
permutation(a)
```

Permutation (new array)

Distributions in random

```
import numpy.random
```

The list of distributions to sample from is quite long, and includes

- beta
- binomial
- chisquare
- exponential
- dirichlet
- gamma
- laplace
- lognormal
- pareto
- poisson
- power